BRITE-FL: Blockchain-based Reputation and Incentive Token Enhanced Federated Learning for IoT Ecosystems

ARTICLE INFO

Keywords:
Blockchain
Federated Learning
Internet of Things (IoT)
Reputation Mechanism
Token Incentives

ABSTRACT

With the rapid development of Internet of Things (IoT) technologies, effectively utilizing distributed data while preserving privacy has become a critical challenge. This paper proposes a secure federated learning framework integrating blockchain and reputation token mechanisms to address data privacy protection, security maintenance, and participant incentivization in IoT environments.The framework leverages blockchain's decentralization and immutability to design a dynamic reputation token mechanism via smart contracts, which rewards participants based on their model training contributions and long-term performance. To mitigate potential malicious behaviors, the proposed approach introduces multi-dimensional anomaly detection models that comprehensively assess nodes' historical behaviors and current contributions. Moreover, the framework implements a role-based participation model, enabling devices with diverse computational capabilities to engage and receive corresponding incentives.Experimental results demonstrate the framework's robustness: when confronted with data pollution and label flipping attacks from up to 30% malicious nodes, the method limits model performance degradation within 5% and achieves a malicious node detection rate exceeding 94%. The innovative reputation evaluation mechanism systematically distinguishes participant behaviors, causing honest nodes' reputations to progressively increase while malicious nodes' reputations exponentially decline with each malicious act. These findings suggest significant potential for secure and efficient federated learning in IoT and edge computing domains.

1. Introduction

The rapid development and widespread application of Internet of Things (IoT) technology have led to an unprecedented scale of data generation and collection. It is predicted that by 2025, the number of global IoT devices will reach 75 billion [1]. These IoT devices, deployed across smart homes, industrial manufacturing, transportation, and healthcare, continuously generate diverse real-time data[2]. Effectively utilizing these distributed data resources has become crucial in advancing Artificial Intelligence (AI) and Machine Learning (ML) technologies. IoT data-driven AI applications are expected to bring revolutionary changes in smart cities, personalized services, and predictive maintenance, promoting economic development and social progress[3–5].

However, two major challenges have emerged as critical bottlenecks limiting the integrated development of IoT and AI. First, the explosive growth of data has raised significant concerns regarding privacy protection and data security. Traditional centralized machine learning paradigms require aggregating dispersed data to a central server for processing, but this approach faces substantial challenges in IoT environments. The data generated by IoT devices is massive in volume, high in dimensionality, and time-sensitive [6], making large-scale cross-network transmission costly and potentially harmful to devices' limited resources [7]. Second, centrally stored user data becomes vulnerable to network attacks, posing severe risks to user privacy[8]. These challenges are further complicated by strict data protection regulations, such as the European Union's General Data Protection Regulation (GDPR) [9], which imposes specific compliance requirements on personal data handling, exacerbating data silos and limiting cross-regional data sharing.

ORCID(s):

Federated Learning (FL) has emerged as a promising solution to these challenges, offering a novel distributed machine learning paradigm that enables collaborative model training without raw data sharing[10]. In the FL framework, participants (such as IoT devices) train models using local data and only share model updates with a central server for aggregation[11], significantly reducing data transmission volume while protecting privacy[12]. However, several critical challenges remain in practical applications. The heterogeneous nature of IoT devices in hardware configurations, computational capabilities, and network conditions creates an imbalanced distribution of capabilities and data quality among participants[13]. Moreover, in open and dynamic IoT environments, the FL process is vulnerable to malicious nodes that may submit false or harmful model updates (as shown in Figure 1), necessitating robust mechanisms for anomaly detection and participant incentivization[14–16].

While existing research has made progress in addressing these issues, current solutions remain inadequate[10]. Many FL frameworks rely heavily on central server coordination, introducing risks of single-point failure and privacy leakage[10, 17, 18]. Furthermore, these studies often overlook the inherent heterogeneity of IoT environments by assuming uniform computational capabilities across participating nodes[19]. Particularly concerning is the lack of sophisticated mechanisms for handling malicious nodes, as existing reputation-based incentive systems fail to effectively address scenarios where high-reputation nodes engage in malicious activities[20].

Blockchain technology, with its decentralized and immutable characteristics, offers a promising approach to addressing these limitations [21]. By incorporating blockchain into federated learning, systems can achieve peer-to-peer

collaboration without third-party intervention while leveraging consensus mechanisms to suppress malicious behavior [22]. Blockchain's distributed ledgers can securely record model updates, while smart contracts can automate reward and punishment mechanisms, enhancing system transparency and credibility[23–25]. Despite these advantages, existing blockchain-FL approaches often treat security, trust assessment, and economic incentives as separate components, making it difficult to establish clear causal relationships between participant behavior and system-wide consequences. Furthermore, most frameworks assume homogeneous device capabilities, limiting their applicability to heterogeneous IoT ecosystems where computational resources vary significantly across participants.

This paper proposes BRITE-FL, a blockchain-based reputation and incentive token enhanced federated learning framework specifically designed for heterogeneous IoT environments. The main contributions of this work are:

1) We present BRITE-FL, a consortium-blockchain-assisted federated learning framework for IoT, where smart contracts and consensus mechanisms provide auditable registration of model updates and automated execution of protocol rules.
2) We specify a dynamic, time-indexed reputation-token scheme that evaluates participants using both current contribution signals and historical behavior, and uses the resulting reputations as a common input to participant selection *and* on-chain settlement, making the evidence-to-decision pathway explicit.
3) We formalize a role separation between training nodes and validation nodes to accommodate IoT heterogeneity; validation results (including consensus outcomes) serve as evidence that feeds reputation evolution and settlement decisions.
4) We incorporate a multi-dimensional malicious behavior identification procedure based on anomaly detection over model updates, and couple detected abnormality evidence to smart-contract-based punishment mechanisms within the protocol workflow.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the preliminaries of Blockchain Technology, Federated Learning, and Isolation Forest. Section 4 details the proposed framework, including the system model, operation workflow, reputation-based participant selection, and token mechanisms. Section 5 evaluates the framework's performance through comprehensive experiments, and Section 6 concludes the paper.

2. Related work

Blockchain-assisted federated learning (BFL), also referred to as blockchain-enabled FL (BC-FL), has been explored to mitigate the reliance on a trusted central coordinator by providing decentralized coordination, immutable logging, and programmable settlement. Recent surveys summarize architectural patterns, security threats, and open challenges when combining blockchain with FL in IoT/edge

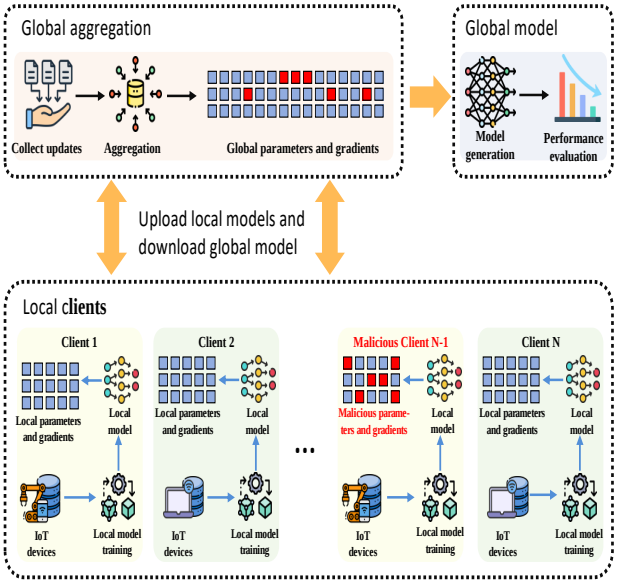

Figure 1: Illustration of FL framework in IoT scenario

environments [1, 3]. Building on these overviews, this section reviews representative studies along three dimensions: (i) decentralized coordination and auditability in IoT, (ii) privacy preservation and robustness, and (iii) incentives and reputation mechanisms.

2.1. Decentralized Coordination and Auditability in IoT

Federated Averaging (FedAvg) [10] is a seminal optimization scheme for FL, where a server coordinates training and aggregates locally computed updates. In cross-organization IoT and edge settings, the coordinator can become a single point of failure and a trusted third party. To reduce this reliance, Lu et al. [26] proposed a permissioned-blockchain-assisted FL architecture for privacy-preserved data sharing in industrial IoT, leveraging the ledger to record model updates and enable traceability. Li et al. [4] further introduced a committee-consensus-based design (BFLC) to improve coordination efficiency when publishing model updates and maintaining the global model in a decentralized manner. Towards fully decentralized coordination, Biscotti [27] co-designs a peer-to-peer training workflow with a blockchain ledger and cryptographic protocols, where committees validate updates and support robust aggregation. Beyond basic coordination, recent studies have investigated efficiency and domain adaptations. Blockchain-aided wireless FL considers resource allocation and client scheduling under communication constraints [28], and IoT-oriented designs further optimize resource usage for blockchain-based FL [7]. Application-specific BFL frameworks have also been studied in industrial IoT, for example, blockchain-based FL for device failure detection [6]. FedTwin [29] extends blockchain-enabled FL to digital-twin networks and introduces an adaptive asynchronous workflow, highlighting the need to cope with heterogeneous and dynamic

industrial assets. Related to emerging workloads, Federated TrustChain [30] explores blockchain-enhanced transparency and unlearning in federated large language model training. These efforts collectively show that blockchain can serve as a coordination and audit layer, while system performance and device heterogeneity remain key considerations.

2.2. Privacy Preservation and Robustness

Although blockchain provides immutability and accountability, confidentiality of local updates and resilience to adversarial behaviors must still be ensured at the learning layer. Differential privacy and cryptographic techniques are common choices. AdaDpFed [31] adaptively calibrates differential privacy (DP) noise for non-IID FL to improve the privacy–utility trade-off against inference attacks. FedML-HE [20] leverages homomorphic encryption to enable aggregation over ciphertext, offering strong confidentiality at the cost of higher computation and communication overhead. For IoT deployments, Kalapaaking et al. [32] combine blockchain with trusted execution environments to realize secure aggregation with auditability, while Wu et al. [33] propose a privacy-preserving serverless FL scheme for IoT, aiming to remove a single coordination point and reduce privacy exposure. Robustness against poisoning and Byzantine clients is another long-standing challenge in open FL. FLForest [34] filters suspicious updates using an isolation-forest-based anomaly scoring mechanism, improving robustness through pre-aggregation screening. Biscotti [27] incorporates committee-based verification and robust aggregation to defend against poisoning attacks in a fully decentralized setting. In parallel, reputation-aware participant selection has been explored for heterogeneous IoT environments [19, 35], indicating that combining statistical robustness with long-term trust assessment can mitigate the impact of unreliable clients. Overall, existing studies provide complementary privacy and robustness tools, but the interaction between (i) privacy protection, (ii) verifiability of update quality, and (iii) decentralized enforcement remains non-trivial in blockchain-assisted IoT settings.

2.3. Incentives and Reputation Mechanisms

Sustaining collaborative training among self-interested parties requires incentives beyond purely technical security mechanisms. Prior work has investigated incentive design using economic modeling, such as contract-theoretic schemes coupled with reputation estimation [36], and hierarchical incentive mechanisms in mobile environments [37]. Blockchain-based approaches offer programmable settlement and transparent accounting. FGFL [8] assesses participants through reputation and contribution indicators to support fair reward distribution and attacker deterrence. FL-Incentivizer [25] tokenizes model assets and governs training/trading via smart contracts, whereas Wu and Seneviratne [38] focus on scalability and incentive compatibility in blockchain-based FL. Further, cross-silo incentive mechanisms and joint resource-allocation designs have been studied in blockchain-based FL [24, 39], illustrating the coupling between system performance and reward allocation. Despite the progress, a recurring limitation is that contribution evaluation, robustness signals, and economic settlement are often handled in separate modules, making it difficult to form an end-to-end workflow where update-quality evidence can consistently drive participant selection, reputation evolution, and automated reward/penalty execution. Motivated by this gap, we design BRITE-FL to integrate multi-source evidence, reputation dynamics, and on-chain settlement for IoT-oriented federated learning.

3. Preliminaries

3.1. Blockchain Technology

Blockchain technology has revolutionized data management and transaction processing by providing a decentralized, immutable, and transparent framework. At its core, a blockchain is a distributed ledger maintained by a network of participants, each holding a copy of the ledger. The ledger is composed of a series of blocks, each containing a set of transactions, linked together using cryptographic hashes to form a tamper-proof chain [23]. In a permissioned blockchain, such as a consortium blockchain, the network is managed by a group of pre-selected participants responsible for validating transactions and appending new blocks to the chain [21]. This type of blockchain is well-suited for applications that require a controlled environment, such as federated learning, where only authorized participants can join the network and contribute to the process.

One of the key features of blockchain technology is the use of smart contracts, which are self-executing programs stored on the blockchain that can be triggered by specific events or conditions [9]. A smart contract can be formally defined as a tuple $SC = (S, F, R)$, where S represents the state variables, F is the set of functions that can be invoked by the participants, and R denotes the rules and conditions governing the execution of the contract [22]. The execution of a smart contract is triggered by a transaction sent to the blockchain, which invokes one of the contract's functions. The transaction is then validated by the network participants through the consensus mechanism, and if deemed valid, the corresponding function is executed, resulting in a change of the contract's state.The combination of blockchain technology and smart contracts offers several advantages, such as increased security, transparency, and automation. By leveraging the security and immutability of the blockchain, along with the self-executing nature of smart contracts, we can create a trustworthy and efficient environment for collaborative model training and aggregation in the context of federated learning. Blockchain and smart contracts can be utilized to ensure the integrity and confidentiality of the learning process, while also providing a mechanism for incentivizing participants to contribute high-quality data and model updates[18].

3.2. Federated Learning

Federated learning (FL) is a distributed machine learning paradigm that enables multiple participants to collaboratively train a shared model while keeping their data locally,

Table 1
System-level comparison of representative *blockchain-assisted FL frameworks*. •: explicitly supported; ∘: partially/implicitly supported; –: not reported/unclear. We only mark features that are explicitly stated in the corresponding paper.

Indicator	[26]	[4]	[27]	[8]	[25]	[32]	**BRITE-FL**
Reputation	–	–	∘	•	–	–	•
Reward / Penalty	–	–	∘	•	•	–	•
Stake / Deposit	–	–	∘	–	–	–	•
Update Verif. / Audit	∘	•	•	∘	∘	∘	•
Malicious Det./ Robust	–	–	•	∘	–	–	•
Hetero. Roles	–	•	•	–	∘	∘	•
Evidence→Selection	–	∘	•	∘	–	–	•
Evidence→Settlement	–	–	∘	•	•	–	•

thereby preserving data privacy [4, 16]. In this setting, a set of participants $\mathcal{P} = \{P_1, P_2, ..., P_N\}$, each holding a private dataset D_i, cooperate to learn a global model with parameters θ. The objective is to minimize the global loss function, which is a weighted average of the local loss functions:

$$\min_{\theta} F(\theta) = \sum_{i=1}^{N} \frac{n_i}{n} F_i(\theta) \tag{1}$$

where N is the number of participants, $n_i = |D_i|$ is the size of the local dataset belonging to participant P_i, $n = \sum_{i=1}^{N} n_i$ is the total number of data samples across all participants, and $F_i(\theta)$ is the local objective function of participant P_i.

The Federated Averaging (FedAvg) algorithm [2] is a popular approach to solve this optimization problem. In each round t of FedAvg, the server selects a random subset of participants $S_t \subseteq \mathcal{P}$ and broadcasts the current global model θ_t to them. The selected participants then perform local training on their respective datasets to update the model parameters:

$$\theta_i^{t+1} = \theta_i^t - \eta \nabla F_i(\theta_i^t) \tag{2}$$

where η is the learning rate. After local training, the participants send their updated models θ_i^{t+1} back to the server, which aggregates them to obtain the new global model:

$$\theta_{t+1} = \sum_{i=1}^{N} \frac{n_i}{n} \theta_i^{t+1} \tag{3}$$

This process is repeated for a predefined number of rounds or until convergence.

Federated learning provides a principled framework for enabling collaborative learning while preserving data privacy. However, it faces challenges such as the potential presence of malicious participants who may attempt to undermine the learning process by sending incorrect or manipulated updates [18]. To mitigate these challenges, various techniques have been proposed to enhance the robustness and security of federated learning systems [19, 20].

3.3. Isolation Forest

Isolation Forest (iForest) is an unsupervised anomaly detection algorithm proposed by Liu et al. [40]. Unlike traditional density or distance-based methods that rely on profiling normal behavior, iForest operates on the principle that anomalous instances are fundamentally different from normal instances and can be more easily separated or "isolated" from the majority of data points [29, 41].

The core mechanism of iForest involves constructing an ensemble of isolation trees (iTrees), where each iTree is built by recursively partitioning the data space through random attribute selection and random split-point determination. The algorithm constructs these trees by randomly selecting a feature and then randomly selecting a split value between the maximum and minimum values of the selected feature. This process continues recursively until each data point is isolated in its own leaf node or a predefined tree height is reached.

The fundamental insight underlying iForest is that anomalies exhibit significantly shorter average path lengths in iTrees compared to normal data points. For a dataset of size n, normal instances require approximately $O(\log n)$ steps to be isolated, as they follow patterns similar to the majority of the data. In contrast, anomalous instances can be isolated with substantially fewer partitions due to their distinct characteristics that differentiate them from the dense regions of normal data.

The anomaly score for each data point x is calculated based on its average path length $E(h(x))$ across multiple iTrees in the ensemble:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}} \tag{4}$$

where $E(h(x))$ represents the average path length of data point x over all iTrees in the forest, and $c(n)$ is the average path length of unsuccessful search in a Binary Search Tree (BST), defined as $c(n) = 2H(n-1) - \frac{2(n-1)}{n}$ for $n > 2$, where $H(i)$ is the harmonic number. This normalization factor enables meaningful comparison of anomaly scores across datasets of different sizes.

The anomaly score $s(x, n)$ ranges from 0 to 1, where values close to 1 indicate strong anomalous behavior, values

significantly smaller than 0.5 suggest normal behavior, and values around 0.5 indicate that the data point cannot be clearly distinguished as normal or anomalous. The ensemble approach, utilizing multiple iTrees with different random partitioning strategies, effectively reduces the impact of randomness and improves the robustness and reliability of anomaly detection.

iForest demonstrates several advantageous properties: linear time complexity with low constant factors, minimal memory requirements, and the ability to handle high-dimensional datasets without requiring distance calculations or density estimations. These characteristics distinguish it from traditional anomaly detection methods such as k-nearest neighbors or support vector machines, which typically suffer from the curse of dimensionality and require substantial computational resources for distance or similarity calculations.

In distributed computing environments, iForest's design supports natural parallelization and distributed processing. Model parameters from machine learning systems can be vectorized by flattening weight matrices and bias vectors into high-dimensional feature vectors, making them amenable to anomaly detection analysis. When multiple validation entities are involved, each can independently construct isolation forests using the same parameter vectors, and their anomaly scores can be aggregated through consensus mechanisms to enhance detection consistency. The algorithm's robustness to noise and its capacity to identify anomalous patterns without requiring prior knowledge of attack signatures make it suitable for detecting malicious model updates, where adversarial modifications often exhibit systematic deviations from legitimate parameter distributions.

4. Proposed protocol

This section proposes a blockchain-based Internet of Things (IoT) federated learning framework designed to promote the participation of high-quality nodes and encourage honest behavior through a reputation token incentive mechanism. The framework primarily consists of four key components: task publishers, participating nodes, blockchain network, and smart contracts. Figure 2 illustrates the overall architecture of the system and the interactions among various components. Table 2 lists the main parameters used in this paper and their definitions.

4.1. System Model

The core components of this framework and their main functions are as follows:

1) Task Publisher (T): As the initiator of the system, responsible for publishing federated learning tasks on the blockchain. Task information includes task objectives (G), data requirements (D_r), model type (M), and total reward tokens (R_{total}). This information is publicly released through the blockchain (B), which ensures that all potential participants can access and understand the task requirements.
2) Participating Nodes: key entities executing federated learning tasks, divided into two roles: training nodes and validation nodes.
   (a) Training Nodes ($n_{ti} \in N_t$): Responsible for providing local data and executing model training. Each training node possesses a local dataset D_{ti}, performs model training based on the global model parameters $\theta_g^{(t-1)}$ and local data, and uploads the model update $\Delta\theta_{ti}^{(t)}$ to the blockchain network.
   (b) Validation Nodes ($n_{vi} \in N_v$): Responsible for validating the effectiveness of training results. Validation nodes retrieve updates $\{\Delta\theta_{tj}^{(t)}\}$ submitted by training nodes and the corresponding $\theta_g^{(t-1)}$ from the blockchain network and evaluate reconstructed candidate models $\theta_j^{(t)} = \theta_g^{(t-1)} + \Delta\theta_{tj}^{(t)}$ using the public validation dataset D_v. Evaluation results are uploaded to the smart contract as scores, used to update reputation values of training nodes.

   Participants can choose their roles based on their resources and willingness. They need to stake a certain amount of tokens C_i during registration. These tokens are locked by the smart contract and converted into initial reputation values $R_{init,i}$, serving as proof of qualification for task participation.
3) Blockchain Network (B): As the system's infrastructure, responsible for storing task information, participant information, and model updates/parameters, and recording key data during each training round. The blockchain network design ensures transparency and traceability of the entire federated learning process, avoiding the risk of single-point failure associated with centralized storage.
4) Smart Contract(SC): An automatically executed program deployed on the blockchain network, responsible for managing the entire federated learning process. Main functions include participant selection, token management, training process coordination, reputation value updates, and reward distribution. The smart contract selects the final participating nodes (N_t' and N_v') from the corresponding candidate sets (N_t and N_v) based on participants' reputation values R_i and staked token amounts C_i. After each training round, the smart contract aggregates the *accepted* updates (i.e., those not marked as malicious by validation) to obtain new global model parameters, i.e., $\theta_g^{(t)} = f(\theta_g^{(t-1)}, \{\Delta\theta_{ti}^{(t)} : \mathcal{M}_i^{(t)} = 0\})$, and distributes $\theta_g^{(t)}$ to training nodes to start the next round.

4.2. System Operation Workflow

Figure 3 illustrates the overall system operation process, detailing the interactions between different components. The entire process can be divided into the following steps:

Step 1. Publish task. The task publisher T releases the requirements for the federated learning task (including the task objective G, data requirements D_r, model type M, and total reward tokens R_{total}) onto the blockchain B, making this information accessible to all potential participants.

Table 2
System Parameters Definition

Parameter	Definition	Parameter	Definition
T	Task Publisher	G	Task Goal
D_r	Data Requirement	M	Model Type
R_{total}	Total Reward Tokens	B	Blockchain Network
SC	Smart Contract	N_t	Set of Candidate Training Nodes
N_v	Set of Candidate Validation Nodes	k_t, k_v	Number of selected training/validation nodes ($\|N'_t\| = k_t$, $\|N'_v\| = k_v$)
N'_t, N'_v	Selected Training/Validation Node Sets	k	Size of per-round validation group ($k \le \|N'_v\|$)
$V^{(t)}$	Validation group in round t, $V^{(t)} \subseteq N'_v$, $\|V^{(t)}\| = k$	n_{ti}	The i-th Training Node
n_{vi}	The i-th Validation Node	D_{ti}	Local Dataset of Training Node n_{ti}
D_v	Public Dataset for Validation	$\theta^{(t)}_{ti,e}$	Local parameters of n_{ti} after e local steps in global round t
$\theta^{(t)}_g$	Global Model Parameters in the t-th Round	$\Delta\theta^{(t)}_{ti}$	Model update uploaded by n_{ti} in round t
L	Loss Function used for model training	f	Aggregation function (FedAvg on accepted updates)
η	Learning rate	E	Number of local update steps/epochs per global round
C_i	Amount of staked tokens by participant i (registration stake)	C_{avg}	Average amount of tokens staked by all participants
R_i	Reputation value of participant i (generic)	$R_{init,i}$	Initial reputation value of participant i
δ_d	Reputation decay factor	$\Delta R_i^{\text{decay}}$	Reputation reduction due to decay in selection stage
$\alpha, \beta, \gamma, \delta_r$	Weight coefficients in the selection score	S_i	Comprehensive selection score of participant i
$R^{(t)}_{ti}, R^{(t)}_{vi}$	Reputation of training node n_{ti} / validation node n_{vi} at round t	R_{min}	Minimum reputation threshold for accepting updates
A^i_j	Accuracy of candidate model from training node j evaluated by validation node i	Z^i_j	Z-score of model accuracy for training node j by validation node i
s^i_j	iForest anomaly score for training node j by validation node i	w_1, w_2	Weight coefficients for Z^i_j and s^i_j in validation scoring
S^i_j	Normalized severity score reported by validation node i for training node j	S_j	Final aggregated severity score of training node j after consensus
τ	Severity threshold for malicious update detection	$\mathcal{M}^{(t)}_j$	Malicious behavior indicator for node j in round t
$N^{(t)}_j$	Cumulative number of detected malicious behaviors by node j up to round t	$\lambda_h, \lambda_m, \kappa$	Training-node reputation update parameters
λ_t, λ_v	Reward ratios for training/validation nodes ($\lambda_t + \lambda_v = 1$)	η_v	Validation-node reputation adjustment factor
T_{round}	Total number of training rounds	$\mathcal{C}_{end}$	Termination conditions
C_{ti}	Contribution score of training node n_{ti} for reward calculation	W_{ti}	Reward for training node n_{ti}
W_{vi}	Reward for validation node n_{vi}	$\bar{R}_i$	Normalized reputation in $[0, 1]$ (used for deposit sizing)
Dep_i	Task deposit locked for participant i (penalty mechanism)	Dep_{min}	Minimum task deposit requirement
μ	Deposit coefficient	ρ_{base}	Base confiscation ratio
ν	Confiscation adjustment coefficient	Sev_i	Severity score used for penalty ($[0, 1]$)
ϵ	A small constant for numerical stability		

Step 2. Participant registration. Participants choose to become either training nodes n_{ti} or validation nodes n_{vi} based on their resources and willingness, and proceed with registration. During the registration process, participants must stake a certain amount of tokens C_i, which will be locked by the smart contract SC and converted into an initial reputation value $R_{init,i}$.

Step 3. Participant election (training & validation). The smart contract SC determines the final node sets N'_t and N'_v to participate in the task from the corresponding candidate sets N_t and N_v, based on the participants' reputation values R_i and staked tokens C_i.

Step 4. Local model training and validation. The selected training nodes $n_{ti} \in N'_t$ perform local training based on the global model parameters $\theta^{(t-1)}_g$ and local datasets D_{ti}, and upload the model updates $\Delta\theta^{(t)}_{ti}$ to the blockchain network B. Subsequently, the validation nodes $n_{vi} \in N'_v$ are assigned

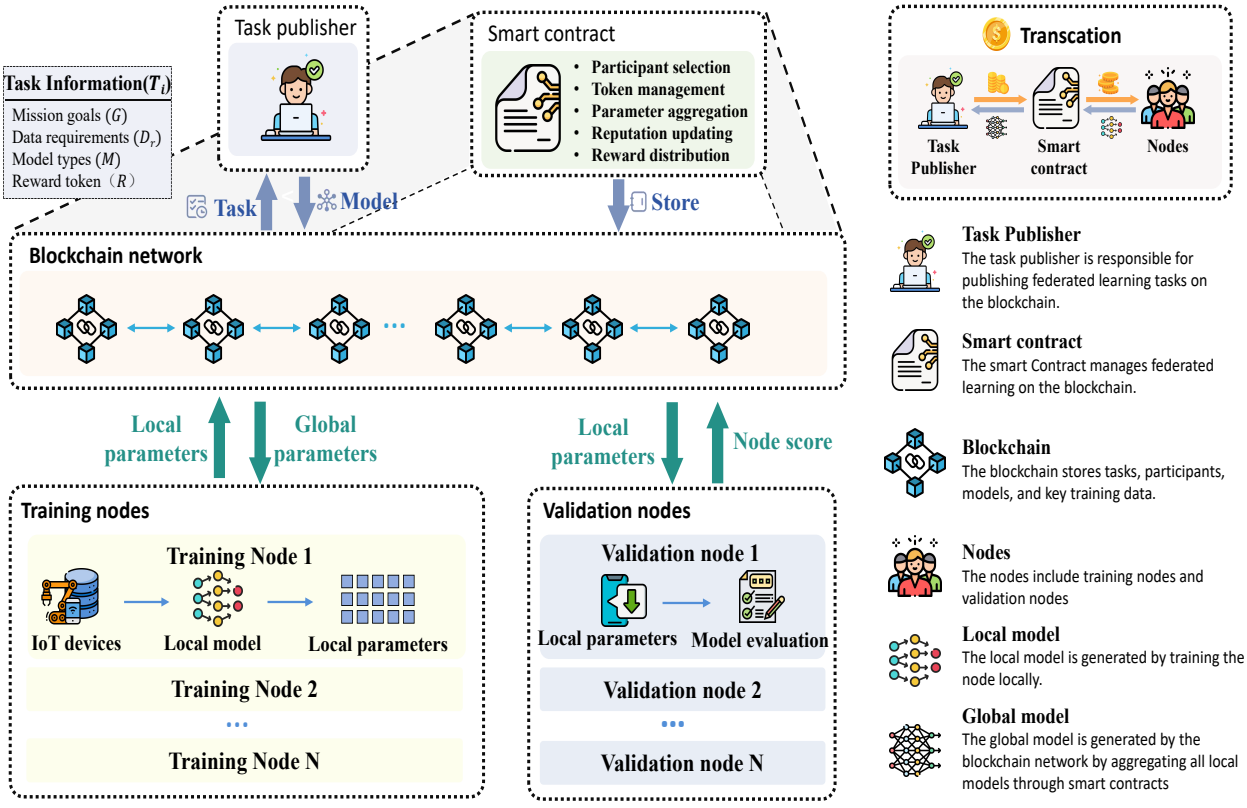

Figure 2: System Model for a Blockchain-based Federated Learning Framework in the Internet of Things (IoT)

into a validation group $V^{(t)}$ and evaluate candidate models reconstructed from $\theta_g^{(t-1)}$ and $\Delta\theta_{ti}^{(t)}$ using the validation dataset D_v. The evaluation scores are fed back to the smart contract, which are used to update the corresponding training nodes' reputation values R_i.

Step 5. Global model aggregation and distribution. Once the smart contract has collected a sufficient number of validated model updates and formed consensus, it executes parameter aggregation over the accepted updates (i.e., $\mathcal{M}_i^{(t)} = 0$) to generate new global model parameters $\theta_g^{(t)} = f(\theta_g^{(t-1)}, \{\Delta\theta_{ti}^{(t)} : \mathcal{M}_i^{(t)} = 0\})$. The aggregated model parameters are then distributed to the training nodes, initiating a new round of training.

Step 6. Iterate the loop. Steps 4–5 are repeated until the predetermined termination conditions C_{end} are met. These conditions may include a preset number of training rounds, a model accuracy threshold, etc.

Step 7. Return the results. The smart contract SC distributes rewards based on participants' contributions and reputation values, and refunds any remaining locked tokens that were not deducted. After task completion, participants can receive token rewards W_{ti} or W_{vi} based on their roles and performance.

4.3. Reputation-Based Participant Selection Management Mechanism

The participant selection and reputation value management mechanism is a key component of the blockchain-based federated learning framework. This mechanism aims to ensure that high-quality participants are selected more often for tasks while effectively constraining the selection of low-quality or malicious participants. Algorithm 1 describes the participant selection process in detail.

First, the node selection mechanism is primarily based on participants' reputation values and the amount of staked tokens. The smart contract calculates participants' comprehensive scores S_i, ranks them, and selects top-ranking nodes (within each role) to participate in the federated learning task.

For participants joining the system, their initial reputation value $R_{init,i}$ is determined by the amount of staked tokens C_i. The formula for calculating the initial reputation value is as follows:

$$R_{init,i} = f(C_i) = \frac{C_i}{C_{avg}} \cdot \left(1 + \log(1 + C_i)\right) \tag{5}$$

where C_{avg} represents the average number of tokens staked by all participants.

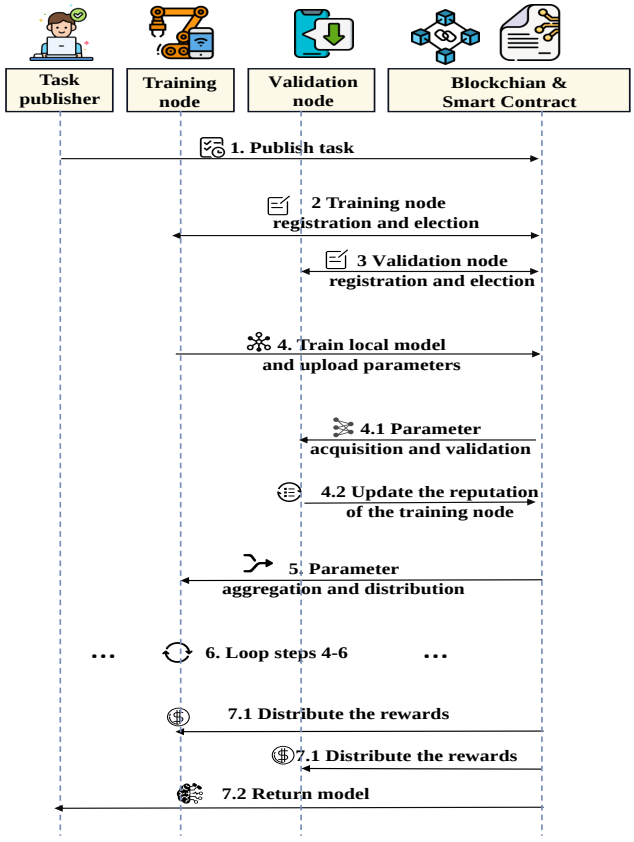

Figure 3: System Workflow Diagram

To prevent high-reputation nodes from monopolizing system resources over extended periods, this study introduces a mechanism for reputation value decay over time:

$$R_{i,t} = R_{i,t-1} \cdot \delta_d, \quad 0 < \delta_d < 1 \tag{6}$$

and the decay-induced change used in selection is:

$$\Delta R_i^{\text{decay}} = R_{i,t-1} - R_{i,t} \tag{7}$$

Finally, the calculation of the comprehensive score S_i considers reputation value R_i, staked tokens C_i, and decay-induced change $\Delta R_i^{\text{decay}}$:

$$S_i = \alpha \cdot R_i + \beta \cdot \left(\frac{C_i}{C_{avg}}\right)^{\gamma} + \delta_r \cdot \Delta R_i^{\text{decay}} \tag{8}$$

After completing the comprehensive score calculation, the system ranks participants by S_i within N_t and N_v, and selects the top-ranking nodes as the final training nodes N_t' and validation nodes N_v'.

4.4. Model Update and Validation

This section elaborates on the node training process and validation mechanism in the blockchain-based federated learning framework with reputation token incentives.

Algorithm 1: Participant Selection and Reputation Management

Input: Candidate training nodes N_t, candidate validation nodes N_v, staked tokens $\{C_i\}$, decay factor δ_d, weight coefficients $\alpha, \beta, \gamma, \delta_r$, numbers of selected nodes k_t, k_v
Output: Selected training nodes N_t', validation nodes N_v'

$N \leftarrow N_t \cup N_v$;
$C_{avg} = \frac{1}{|N|} \sum_{i \in N} C_i$;
for *each node* $i \in N$ **do**
  if *i is a new node* **then**
    $R_{init,i} = \frac{C_i}{C_{avg}} \cdot (1 + \log(1 + C_i))$;
    $R_i \leftarrow R_{init,i}$;
    $\Delta R_i^{\text{decay}} \leftarrow 0$;
  else
    $R_i^{old} \leftarrow R_i$;
    $R_i \leftarrow R_i^{old} \cdot \delta_d$;
    $\Delta R_i^{\text{decay}} \leftarrow R_i^{old} - R_i$;
  $S_i = \alpha \cdot R_i + \beta \cdot (\frac{C_i}{C_{avg}})^{\gamma} + \delta_r \cdot \Delta R_i^{\text{decay}}$;
Sort nodes in N_t in descending order based on S_i;
$N_t' =$ Top k_t nodes from sorted N_t;
Sort nodes in N_v in descending order based on S_i;
$N_v' =$ Top k_v nodes from sorted N_v;
return N_t', N_v';

Algorithm 2: Local Model Update (in global round t)

Input: Training node set N_t', local datasets $\{D_{ti}\}$, learning rate η, local update steps E, reputation threshold R_{min}
Output: Uploaded updates $\{\Delta\theta_{ti}^{(t)}\}$

for *each* $n_{ti} \in N_t'$ **do**
  Preprocess local dataset D_{ti};
  Retrieve global model parameters $\theta_g^{(t-1)}$ from blockchain B;
  Initialize local model $\theta_{ti,0}^{(t)} \leftarrow \theta_g^{(t-1)}$;
  for $e = 1$ **to** E **do**
    $\theta_{ti,e}^{(t)} = \theta_{ti,e-1}^{(t)} - \eta \nabla L(\theta_{ti,e-1}^{(t)}, D_{ti})$;
  $\Delta\theta_{ti}^{(t)} \leftarrow \theta_{ti,E}^{(t)} - \theta_g^{(t-1)}$;
  if $R_{ti}^{(t-1)} \geq R_{min}$ **then**
    Upload $\Delta\theta_{ti}^{(t)}$ to blockchain B;
  else
    Reject $\Delta\theta_{ti}^{(t)}$;
    Record node's behavior;
return $\{\Delta\theta_{ti}^{(t)}\}$;

4.4.1. Local Model Update

When the smart contract SC determines the final set of training nodes N_t', training nodes $n_{ti} \in N_t'$ execute the local update procedure in each global round t as shown in Algorithm 2:

1) Data Preparation and Model Initialization: Each training node n_{ti} prepares its local dataset D_{ti} and preprocesses the data. At the beginning of global round t, training nodes retrieve the latest global model parameters $\theta_g^{(t-1)}$ from the blockchain B and initialize their local model parameters $\theta_{ti,0}^{(t)}$ with $\theta_g^{(t-1)}$.
2) Model Training and Parameter Update: Training nodes perform E local optimization steps/epochs on D_{ti}:

$$\theta_{ti,e}^{(t)} = \theta_{ti,e-1}^{(t)} - \eta \nabla L(\theta_{ti,e-1}^{(t)}, D_{ti}), \quad e = 1, \dots, E \tag{9}$$

3) Update Upload: After completing E local steps in round t, training nodes upload the update:

$$\Delta\theta_{ti}^{(t)} = \theta_{ti,E}^{(t)} - \theta_g^{(t-1)} \tag{10}$$

The smart contract checks the reputation value $R_{ti}^{(t-1)}$ of the submitting node. If $R_{ti}^{(t-1)} < R_{min}$, the smart contract rejects the submitted update and records the node's behavior:

$$\text{Accept } \Delta\theta_{ti}^{(t)} \text{ if } R_{ti}^{(t-1)} \geq R_{min}, \text{ else reject} \tag{11}$$

4.4.2. Validation and Reputation Management

Validation nodes play a crucial role in this framework. In this section, S_j^i and S_j are interpreted as normalized *severity* (suspiciousness) scores in $[0, 1]$, where a larger value indicates a more suspicious update.

The validation process is as follows:

1) Validation Node Assignment: Randomly select k validation nodes from N_v' to form a validation group $V^{(t)} = \{v_1, v_2, ..., v_k\}$. Typically, $k \leq |N_v'| = k_v$.
2) Independent Validation: Each validation node $v_i \in V^{(t)}$ independently executes the following steps:
   a) Retrieve the submitted updates $\{\Delta\theta_j^{(t)}\}$ and global model $\theta_g^{(t-1)}$ from the blockchain B, and reconstruct candidate models $\theta_j^{(t)} = \theta_g^{(t-1)} + \Delta\theta_j^{(t)}$.
   b) Calculate the accuracy of each candidate model using the validation dataset D_v:

$$A_j^i = \text{Accuracy}(\theta_j^{(t)}, D_v), \quad j \in N_t' \tag{12}$$

   c) Perform Z-score standardization on accuracies:

$$Z_j^i = \frac{A_j^i - \mu_i}{\max(\sigma_i, \epsilon)} \tag{13}$$

where μ_i and σ_i are the mean and standard deviation of $\{A_j^i\}_{j \in N_t'}$, and ϵ is a small constant for stability.

Algorithm 3: Validation Process

Input: Validation group $V^{(t)} = \{v_1, ..., v_k\} \subseteq N_v'$, global model $\theta_g^{(t-1)}$, submitted updates $\{\Delta\theta_j^{(t)}\}$, validation dataset D_v, weights w_1, w_2, severity threshold τ

Output: Final scores $\{S_j\}$, malicious behavior indicators $\{\mathcal{M}_j^{(t)}\}$

Initialize $\{\mathcal{M}_j^{(t)}\} = 0$ for all training nodes;
for *each* $v_i \in V^{(t)}$ **do**
  Retrieve $\theta_g^{(t-1)}$ and $\{\Delta\theta_j^{(t)}\}$ from blockchain B;
  Train (or update) an isolation forest $\mathcal{F}_t$ using $\{\Delta\theta_j^{(t)}\}$;
  for *each training node* $j \in N_t'$ **do**
    $\theta_j^{(t)} \leftarrow \theta_g^{(t-1)} + \Delta\theta_j^{(t)}$;
    $A_j^i = \text{Accuracy}(\theta_j^{(t)}, D_v)$;
    $s_j^i = 2^{-\frac{\text{E}[h(\Delta\theta_j^{(t)})]}{c(|N_t'|)}}$;
  $(\mu_i, \sigma_i) = \text{MeanAndStdDev}(\{A_j^i\}_{j \in N_t'})$;
  for *each training node* $j \in N_t'$ **do**
    $Z_j^i = \frac{A_j^i - \mu_i}{\max(\sigma_i, \epsilon)}$;
    $\tilde{S}_j^i = w_1(-Z_j^i) + w_2 s_j^i$;
    $S_j^i = \text{Clip}_{[0,1]}(\text{Normalize}(\tilde{S}_j^i))$;
for *each training node* $j \in N_t'$ **do**
  $(S_j^{\text{median}}, \sigma_{S_j}) = \text{MedianAndStdDev}(\{S_j^i\}_{v_i \in V^{(t)}})$;
  $S_j^{\text{valid}} = \{S_j^i : |S_j^i - S_j^{\text{median}}| \leq 2\sigma_{S_j}\}$;
  if $|S_j^{valid}| \geq \lceil 2k/3 \rceil$ **then**
    $S_j = \text{Average}(S_j^{\text{valid}})$;
    if $S_j > \tau$ **then**
      $\mathcal{M}_j^{(t)} = 1$;
  else
    Reassign $V^{(t)}$ from N_v' and repeat validation;
return $\{S_j\}, \{\mathcal{M}_j^{(t)}\}$;

   d) Execute isolation forest anomaly detection and calculate anomaly scores:

$$s_j^i = s(\Delta\theta_j^{(t)}, \mathcal{F}_t) = 2^{-\frac{\text{E}[h(\Delta\theta_j^{(t)})]}{c(|N_t'|)}} \tag{14}$$

where $h(\Delta\theta_j^{(t)})$ represents the average path length of update $\Delta\theta_j^{(t)}$ in the isolation forest $\mathcal{F}_t$, and $c(\cdot)$ is a constant related to the sample size (here, the number of updates).

e) Calculate normalized severity scores:

$$\tilde{S}_j^i = w_1(-Z_j^i)+w_2 s_j^i, \qquad S_j^i = \mathrm{Clip}_{[0,1]}(\mathrm{Normalize}(\tilde{S}_j^i)) \tag{15}$$

3) Consistency Check: The smart contract SC calculates the median score S_j^{median} and score standard deviation σ_{S_j} for each training node j. If $|S_j^i - S_j^{\text{median}}| > 2\sigma_{S_j}$, the score is discarded.
4) Consensus Formation: If the number of remaining valid scores is $\geq \lceil 2k/3 \rceil$, the average of these scores is taken as the final score S_j for training node j. Otherwise, the validation round fails; validation nodes are reassigned from N_v' and the validation process is repeated.
5) Validation Node Reputation Update: Based on the proximity of provided scores to the final scores, update the reputation value of validation nodes:

$$R_{vi}^{(t+1)} = R_{vi}^{(t)} + \eta_v \left(1 - \frac{1}{|N_t'|} \sum_{j \in N_t'} \frac{|S_j^i - S_j|}{\max(S_j, \epsilon)}\right) \tag{16}$$

The reputation value $R_j^{(t+1)}$ of training node j after the t-th round is updated as:

$$R_j^{(t+1)} = R_j^{(t)} + \Delta R_j^{(t)} \tag{17}$$

where $\Delta R_j^{(t)}$ is defined as:

$$\Delta R_j^{(t)} = \lambda_h(1 - S_j) - \lambda_m(e^{\kappa N_j^{(t)}} - 1)S_j \tag{18}$$

and the cumulative malicious count is:

$$N_j^{(t)} = N_j^{(t-1)} + \mathcal{M}_j^{(t)}, \quad N_j^{(0)} = 0. \tag{19}$$

This model aims to allow honest nodes' reputations to gradually increase, while malicious nodes' reputations decrease exponentially with the number of malicious acts.

Algorithm 4: Reputation Update Mechanism

Input: Current reputations $\{R_j^{(t)}\}, \{R_{vi}^{(t)}\}$, scores $\{S_j\}, \{S_j^i\}$, malicious behavior indicators $\{\mathcal{M}_j^{(t)}\}$, parameters $\lambda_h, \lambda_m, \kappa, \eta_v$

Output: Updated reputations $\{R_j^{(t+1)}\}, \{R_{vi}^{(t+1)}\}$

for *each training node* j **do**
  $N_j^{(t)} = N_j^{(t-1)} + \mathcal{M}_j^{(t)}$;
  $\Delta R_j^{(t)} = \lambda_h(1 - S_j) - \lambda_m(e^{\kappa N_j^{(t)}} - 1)S_j$;
  $R_j^{(t+1)} = R_j^{(t)} + \Delta R_j^{(t)}$;
for *each validation node* v_i **do**
  $R_{vi}^{(t+1)} = R_{vi}^{(t)} + \eta_v \left(1 - \frac{1}{|N_t'|} \sum_{j \in N_t'} \frac{|S_j^i - S_j|}{\max(S_j, \epsilon)}\right)$;
return $\{R_j^{(t+1)}\}, \{R_{vi}^{(t+1)}\}$;

4.5. Token Reward Distribution and Penalty Mechanism

This section introduces the token reward distribution and penalty mechanism in the blockchain-based federated learning framework with reputation token incentives.

At the beginning of the task, the publisher sets the total reward token amount R_{total}, and specifies the reward ratio λ_t for training nodes and λ_v for validation nodes, where $\lambda_t, \lambda_v \in (0, 1)$ and $\lambda_t + \lambda_v = 1$. Based on this, the training node reward pool is defined as $R_{train} = \lambda_t \cdot R_{total}$ and the validation node reward pool as $R_{verify} = \lambda_v \cdot R_{total}$. Deposits confiscated due to the penalty mechanism will be redistributed to the corresponding reward pools according to the ratio of λ_t and λ_v.

The reward calculation for training nodes is based on their contribution to the model and reputation value:

$$W_{ti} = R_{train} \cdot \frac{C_{ti} \cdot R_{ti}}{\sum_j (C_{tj} \cdot R_{tj})} \tag{20}$$

where R_{ti} reflects the node's historical performance, and the contribution score is defined as:

$$C_{ti} = \frac{1}{T_{round}} \sum_{r=1}^{T_{round}} \frac{\|\Delta\theta_{ti}^{(r)}\|_2}{\|\theta_g^{(r)} - \theta_g^{(r-1)}\|_2} \tag{21}$$

The reward for validation nodes is similarly based on their consistency and reputation value:

$$W_{vi} = R_{verify} \cdot \frac{A_{vi} \cdot R_{vi}}{\sum_j (A_{vj} \cdot R_{vj})} \tag{22}$$

Validation consistency A_{vi} is calculated by comparing the score provided by a validation node with the average scores from all validation nodes:

$$A_{vi} = 1 - \frac{1}{|N_t'|} \sum_{j \in N_t'} |S_j^i - \overline{S}_j| \tag{23}$$

where $\overline{S}_j = \frac{1}{|N_v'|} \sum_{k \in N_v'} S_j^k$.

This framework introduces a task deposit mechanism. Participating nodes lock a task deposit Dep_i (managed by SC) for penalty enforcement. For clarity, we denote the deposit separately from the registration stake, while both can be locked and managed by the same smart contract in implementation. The deposit size is determined based on the task scale and the node's historical behavior:

$$\mathrm{Dep}_i = \max(\mathrm{Dep}_{min}, \mu \cdot R_{total} \cdot (1 - \bar{R}_i)) \tag{24}$$

where $\bar{R}_i \in [0, 1]$ is the normalized reputation. In this work, we use a simple normalization:

$$\bar{R}_i = \min\left(1, \frac{R_i}{R_{max}}\right), \quad R_{max} = \max_{j \in (N_t \cup N_v)} R_j. \tag{25}$$

When malicious behavior is detected, the system will confiscate part or all of the deposit. The confiscation ratio

ρ is dynamically adjusted according to the severity of the behavior:

$$\rho = \min(1, \rho_{base} + \nu \cdot \mathrm{Sev}_i) \tag{26}$$

where ρ_{base} is the base confiscation ratio, ν is the adjustment coefficient, and $\mathrm{Sev}_i \in [0, 1]$ is the severity score. In particular, for a detected malicious training node j, we set $\mathrm{Sev}_j \triangleq S_j$ (the final aggregated severity score from Algorithm 3).

Algorithm 5: Token Reward Distribution

Input: Total reward R_{total}, reward ratios λ_t, λ_v, training nodes N_t', validation nodes N_v', submitted updates $\{\Delta\theta_{ti}^{(t)}\}$, global parameters $\{\theta_g^{(t)}\}$, node reputations $\{R_{ti}\}, \{R_{vi}\}$, validation scores $\{S_j^i\}$, total rounds T_{round}

Output: Node rewards $\{W_{ti}\}, \{W_{vi}\}$

$R_{train} = \lambda_t \cdot R_{total}$; $R_{verify} = \lambda_v \cdot R_{total}$;

for *each* $n_{ti} \in N_t'$ **do**

  $C_{ti} = \frac{1}{T_{round}} \sum_{r=1}^{T_{round}} \frac{\|\Delta\theta_{ti}^{(r)}\|_2}{\|\theta_g^{(r)} - \theta_g^{(r-1)}\|_2}$;

  $W_{ti} = R_{train} \cdot \frac{C_{ti} \cdot R_{ti}}{\sum_j (C_{tj} \cdot R_{tj})}$;

for *each training node* $j \in N_t'$ **do**

  $\overline{S}_j = \frac{1}{|N_v'|} \sum_{k \in N_v'} S_j^k$;

for *each* $n_{vi} \in N_v'$ **do**

  $A_{vi} = 1 - \frac{1}{|N_t'|} \sum_{j \in N_t'} |S_j^i - \overline{S}_j|$;

  $A_{vi} \leftarrow \max(0, \min(1, A_{vi}))$;

  $W_{vi} = R_{verify} \cdot \frac{A_{vi} \cdot R_{vi}}{\sum_j (A_{vj} \cdot R_{vj})}$;

return $\{W_{ti}\}, \{W_{vi}\}$;

Algorithm 6: Deposit and Penalty Mechanism

Input: Total reward R_{total}, minimum deposit Dep_{min}, deposit coefficient μ, normalized reputations $\{\bar{R}_i\}$, base confiscation ratio ρ_{base}, adjustment coefficient ν, severity scores $\{\mathrm{Sev}_i\}$

Output: Required deposits $\{\mathrm{Dep}_i\}$, confiscation ratios $\{\rho_i\}$

for *each node* $i \in N_t' \cup N_v'$ **do**

  $\mathrm{Dep}_i = \max(\mathrm{Dep}_{min}, \mu \cdot R_{total} \cdot (1 - \bar{R}_i))$;

for *each detected malicious node* i **do**

  $\rho_i = \min(1, \rho_{base} + \nu \cdot \mathrm{Sev}_i)$;

  Confiscate $\rho_i \cdot \mathrm{Dep}_i$ tokens;

  Redistribute confiscated tokens to reward pools;

return $\{\mathrm{Dep}_i\}, \{\rho_i\}$;

5. Performance Analysis

This section provides a detailed description of the experimental evaluation process for our proposed reputation-based federated learning scheme. We designed a series of experiments to assess the framework's performance in terms of model performance, malicious node identification rate, and changes in node reputation.

5.1. Experimental Environment and Configuration

The hardware environment used for the experiments includes a workstation equipped with an Intel i7-10700K CPU, 32GB RAM, and an NVIDIA RTX 3080 GPU, running Ubuntu 20.04 LTS as the operating system. The programming environment for the experiments is Python 3.8, primarily utilizing the PyTorch framework for model training and testing. The blockchain component simulation was implemented using Hyperchain 2.12.0[42], managed with Docker containers to simulate the distributed ledger operating environment. The underlying consensus mechanism employed is the Robust Byzantine Fault Tolerance (RBFT) algorithm, which ensures consistency and liveness even in the presence of malicious nodes. In our setup, the committee consists of a subset of validation nodes that participate in the consensus process, responsible for ordering transactions and generating blocks. The validation nodes not only evaluate model updates but also act as consensus nodes to maintain the integrity of the ledger. Table 3 lists the configuration of various parameters in our scheme as well as the experimental environment setup. We conducted experiments on the MNIST and Fashion-MNIST (FMNIST) datasets, both comprising 60,000 training and 10,000 test samples. Each image is a 28×28 grayscale representation of a digit (MNIST) or a fashion item (FMNIST). Prior to training, all pixel values were normalized to the range [0,1] without additional data augmentation. We randomly and uniformly distributed the training data across 100 training clients, ensuring that each client received an equal portion of the training set. An additional 50 validation clients were designated to evaluate the submitted model parameters.For the model, we adopted a standard Convolutional Neural Network (CNN) that includes two convolutional-pooling layers followed by two fully connected layers, a common lightweight architecture suitable for MNIST-like datasets. The training process used the Adam optimizer with an initial learning rate of 0.01 and a batch size of 32. We ran a total of 250 training epochs, where in each global communication round, clients performed local updates on their assigned data before uploading model parameters for aggregation.

5.2. Comparison Schemes

To comprehensively evaluate the performance of our proposed scheme, we compared it with the following four approaches:

1) Privacy-Preserving Serverless FL for IoT (PSFL-IoT)[33]: This scheme proposes a serverless federated learning

Table 3
Experimental Environment and Key Parameter Configurations

Parameter	Configuration/Value
Hardware Environment	Intel i7-10700K CPU, 32GB RAM, NVIDIA RTX 3080 GPU
Operating System	Ubuntu 20.04 LTS
Programming Environment	Python 3.8, PyTorch
Blockchain Platform	Hyperchain 2.12.0
Blockchain Consensus Mechanism	RBFT
Dataset	MNIST, Fashion-MNIST (60,000 train / 10,000 test)
Image Size	28×28 grayscale
Number of Training Clients	100
Number of Validation Clients	50
Model Architecture	2 convolution-pooling layers + 2 fully connected layers
Optimizer	Adam
Initial Learning Rate	0.01
Batch Size	32
Total Training Epochs	250
Loss Function	Cross-Entropy
Reputation Update Params ($\lambda_v, \lambda_h, \lambda_m, \kappa, R_{min}$)	(0.05, 0.05, 0.1, 0.15, 3)
Reputation Weights ($\alpha, \beta, \gamma, \delta_r, \delta_d$)	(0.4, 0.3, 0.5, 0.3, 0.95)
Deposit Params ($C_{min}, \mu, \rho_{base}, \nu$)	(100, 0.2, 0.5, 0.1)
Scoring Weights (w_1, w_2)	(0.6, 0.4)

framework for IoT environments based on secure multi-party computation. The approach eliminates central server dependencies, achieves fault tolerance through secret sharing, and provides formal security guarantees against collusion attacks while maintaining accuracy without additional loss.

2) Personalized Federated Learning with Client Reputation (PFL-CR)[35]: This scheme employs a reputation-based approach for traffic flow prediction using Graph Neural Networks and Gated Recurrent Units. It incorporates client reputation mechanisms to defend against malicious attacks while optimizing personalized model training through reputation-guided aggregation strategies.
3) Adaptive Differential Privacy Federated Learning (ADPFL)[31]: This scheme protects data privacy by introducing noise to the uploaded model parameters, adaptively adjusting noise levels to balance model performance and privacy protection.
4) Federated Learning in IoT with Malicious Nodes (FedIoT-Malicious): This baseline represents standard federated learning implementations in IoT environments following the FedIoTs framework[33]. We introduce malicious participants in this setting to evaluate system robustness under adversarial conditions, providing a representative baseline for assessing the effectiveness of security-enhanced federated learning approaches.

Regarding attack scenarios, we rigorously defined the attack parameters to ensure reproducibility. For **data pollution attacks**, malicious clients replace 100% of their local training samples with Gaussian noise ($\mu = 0, \sigma = 1$), effectively submitting random gradients. For **label flipping attacks**, malicious nodes flip the label l of every training sample to $9 - l$ (e.g., $0 \rightarrow 9, 1 \rightarrow 8$). Attacks are "always-on", meaning malicious nodes perform these actions in every round they are selected. We set malicious client proportions at 10%, 20%, and 30

5.3. Experimental Results and Analysis

5.3.1. Model Performance Comparison

First, we compared the global model performance of our proposed framework with baseline schemes in the absence of malicious nodes. Table 4 shows the accuracy of each scheme on MNIST and FMNIST datasets.

As shown in Table 4, the PSFL-IoT scheme achieves 92.8% accuracy on the MNIST dataset and 78.9% on the FMNIST dataset, demonstrating the effectiveness of serverless architecture in maintaining model performance. PFL-CR, which employs reputation-based personalized federated learning, achieved 91.6% and 77.8% accuracy on MNIST and FMNIST respectively, showing competitive performance with its personalized approach. ADPFL, incorporating differential privacy, demonstrated the lowest accuracy at 90.7% and 76.2% for MNIST and FMNIST, respectively, due to the noise introduced to preserve privacy. In contrast, our proposed scheme achieved 92.4% and 78.3% accuracy for MNIST and FMNIST datasets, respectively, indicating minimal compromise in model performance while incorporating additional security mechanisms. These results confirm that our framework's reputation-based selection and anomaly detection mechanisms do not adversely affect performance in benign environments.

However, when malicious nodes participate and submit harmful model updates, the performance of each scheme shows significant differences. Figure 4 and Figure5 show the experimental results of each scheme under data pollution

Table 4
Model Accuracy of Different Schemes Without Malicious Nodes

Scheme	MNIST	FMNIST
PSFL-IoT[33]	92.8%	78.9%
ADPFL[31]	90.7%	76.2%
PFL-CR[35]	91.6%	77.8%
Proposed Scheme	92.4%	78.3%

attack scenarios on MNIST and FMNIST datasets, respectively.

In data pollution attacks, malicious nodes inject corrupted or completely randomized data to degrade the global model's performance. The experimental results reveal distinct vulnerability patterns across different schemes. On the MNIST dataset, FedIoT-Malicious experiences performance degradation under data pollution attacks, with accuracy dropping from 92.8% to 88%, 75%, and 66.9% under 10%, 20%, and 30% malicious node proportions respectively—representing performance losses of 5.2%, 19.2%, and 27.9%. The serverless architecture, while providing privacy guarantees, lacks specialized mechanisms for identifying malicious participants. PFL-CR demonstrates improved resilience, achieving 89.3%, 76.8%, and 68.5% accuracy under the same attack intensities, effectively utilizing its reputation mechanisms to defend against malicious participants.

ADPFL exhibits mixed performance under data pollution attacks. While it achieves 85.2% accuracy with 10% malicious nodes, it performs better than FedIoT-Malicious at 20% malicious nodes (79% vs 75%), likely due to the noise injection masking some attack effects. However, at 30% malicious nodes, ADPFL suffers the most severe degradation (58.9%), as the combination of attack effects and differential privacy noise compounds the performance deterioration. In contrast, our proposed scheme maintains stability with accuracies of 92.1%, 90.8%, and 88.7%, corresponding to minimal degradations of only 0.8%, 2.1%, and 4.0% respectively.

The FMNIST dataset presents more challenging attack scenarios due to its higher visual complexity compared to MNIST. FedIoT-Malicious shows more significant accuracy drops from 78.9% to 73.7%, 66%, and 62.5% (degradations of 6.6%, 16.4%, and 20.8%), indicating that serverless architectures face greater challenges in complex data environments. PFL-CR achieves better resilience with accuracies of 74.9%, 67.4%, and 64.1% (degradations of 3.7%, 13.4%, and 17.6%), confirming that its malicious attack defense capabilities provide consistent protection across different data complexities. ADPFL shows degradations of 6.7%, 17.2%, and 21.3% respectively, with performance becoming increasingly unstable as attack intensity grows. Our proposed scheme maintains superior stability with accuracies of 77.4%, 75.5%, and 74.7% (degradations of only 1.1%, 3.6%, and 4.6%), highlighting the robustness of security-oriented reputation design.

In label flipping attacks, malicious nodes intentionally mislabel part of the data to mislead the model. Figure 6 and Figure 7 show the experimental results under label flipping attack scenarios on MNIST and FMNIST datasets, respectively. On the MNIST dataset, FedIoT-Malicious achieves test accuracies of 82.4%, 76.7%, and 66.2% under 10%, 15%, and 20% malicious node proportions. The serverless architecture's distributed nature makes it challenging to detect coordinated label manipulation attacks across multiple nodes. PFL-CR performs better at 84.1%, 78.5%, and 68.9% respectively, leveraging its reputation mechanism to identify and mitigate the impact of malicious clients, though its approach primarily targets traffic flow prediction scenarios. ADPFL shows the poorest performance at 72.3%, 68.6%, and 60.1%, as the added noise amplifies the confusion caused by incorrect labels. Our scheme maintains test accuracies of 91.8%, 90.4%, and 88.6% in these three scenarios, benefiting from its specialized anomaly detection and security-focused reputation updates.

On the FMNIST dataset, the impact of label flipping attacks becomes more pronounced due to the increased complexity of fashion item classification. FedIoT-Malicious achieves test accuracies of 70.5%, 66.7%, and 58.3% under 10%, 15%, and 20% malicious node proportions, showing larger performance degradation compared to MNIST. PFL-CR performs slightly better at 72.1%, 68.9%, and 60.8% respectively, demonstrating that its malicious attack defense mechanisms provide improved resilience across different data complexities. ADPFL maintains similar performance patterns at 72.3%, 68.6%, and 60.1%, with the differential privacy mechanism showing consistent but limited effectiveness against label manipulation. Our scheme maintains test accuracies of 77.0%, 75.9%, and 73.2% in these scenarios, demonstrating that security-oriented design principles become increasingly important as data complexity grows.

5.3.2. Malicious Node Identification Rate

Beyond model performance evaluation, we assess the framework's capability to identify and isolate malicious participants. Our detection mechanism operates by monitoring reputation score changes and anomaly patterns in model updates. A node is flagged as malicious when its reputation falls below a threshold of 3.0 or exhibits consistent anomalous behavior patterns over multiple rounds. For comparison, PFL-CR identifies dishonest clients based on their credibility evaluation and removes clients whose data quality significantly impacts the personalized model

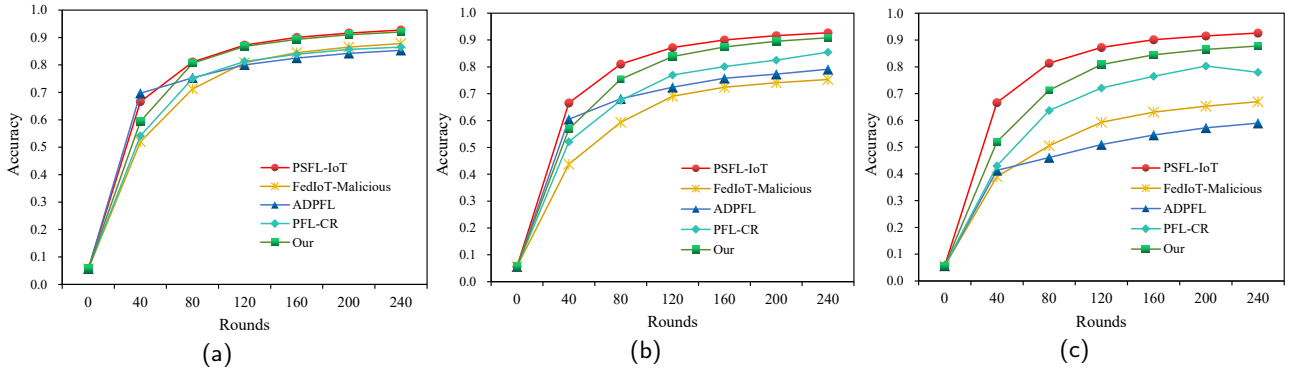

Figure 4: Comparison of Model Performance Under Data Pollution Attacks on MNIST Dataset.
(a)Malicious node proportion: 10% .(b) Malicious node proportion: 20%.(c)Malicious node proportion: 30%

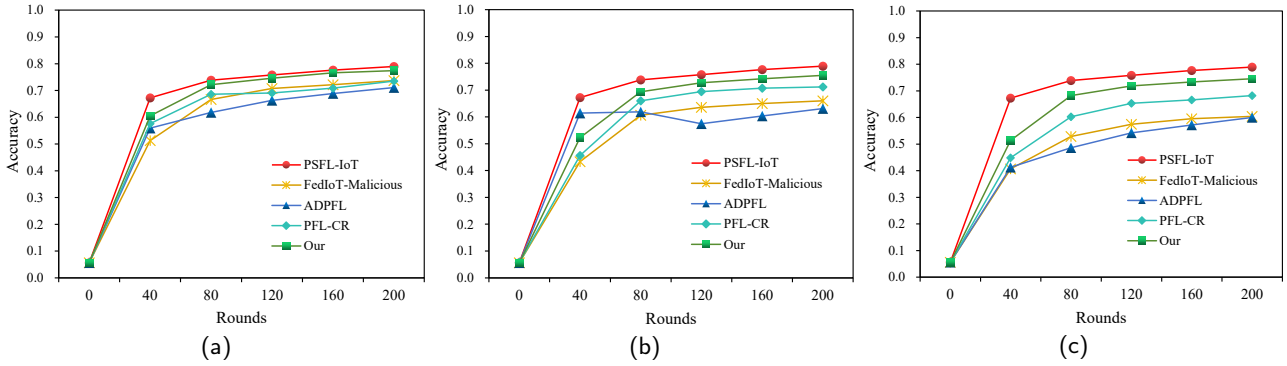

Figure 5: Comparison of Model Performance Under Data Pollution Attacks on FMNIST Dataset.
(a)Malicious node proportion: 10% .(b) Malicious node proportion: 20%.(c)Malicious node proportion: 30%

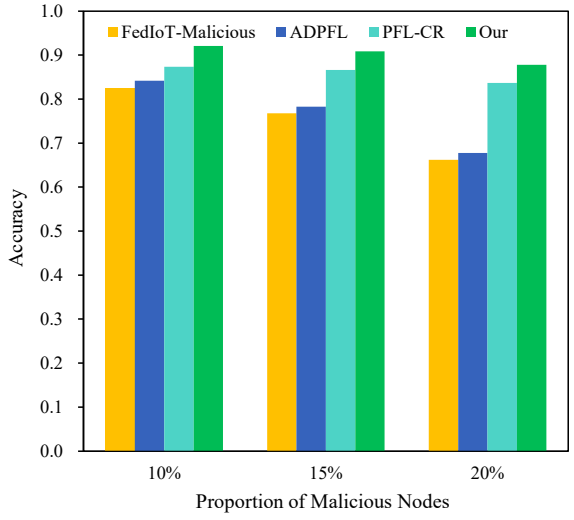

Figure 6: Comparison of Model Performance Under Label Flipping Attacks on MNIST Dataset

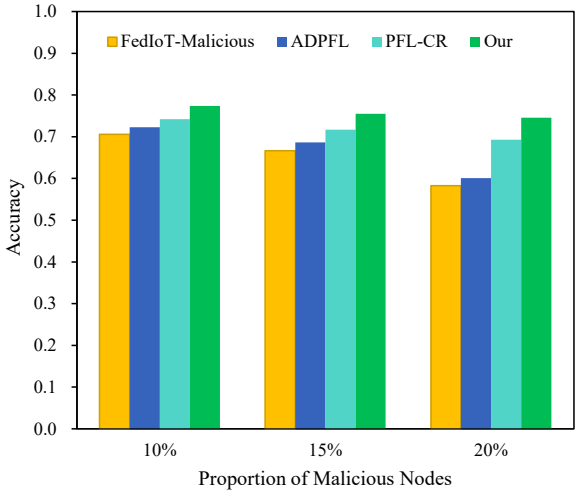

Figure 7: Comparison of Model Performance Under Label Flipping Attacks on FMNIST Dataset

performance. Table 5 presents the detection performance comparison between the two reputation-based schemes.

As shown in Table 5, our proposed framework achieves detection rates above 94% across all attack scenarios, while PFL-CR achieves detection rates ranging from 68-85%. Both schemes utilize reputation mechanisms to identify and handle problematic participants, but differ in their detection sophistication and security focus.

Our detection mechanism combines reputation-based evaluation with anomaly detection and blockchain-based tracking. When malicious participants repeatedly submit

Table 5
Malicious Node Detection Rate Comparison

Attack Type	Malicious	Proposed Scheme	PFL-CR
Data Poisoning	10%	98.5%	85.3%
	20%	97.8%	78.9%
	30%	96.8%	72.4%
Label Flipping	10%	96.2%	76.8%
	15%	95.4%	71.5%
	20%	94.1%	68.2%

harmful updates, their reputation values decline, leading to their eventual removal from training rounds. The multi-layered approach enables robust identification of various attack types including coordinated data poisoning and label flipping scenarios. PFL-CR also employs reputation-based client evaluation and can successfully identify dishonest participants, but our framework incorporates additional security-oriented features such as anomaly pattern analysis and blockchain-based tamper-resistant tracking, enabling more comprehensive threat detection in security-critical environments.

5.3.3. Reputation Value Change Analysis

To analyze the dynamic changes of node reputation values, we recorded the reputation value change curves of typical honest nodes and malicious nodes during the training process. In our experiment, we set the initial reputation value of all nodes to 10 for better comparison. Figure 8 shows the reputation value changes of honest and malicious nodes under 20% data pollution attacks in the MNIST experiment.

Figure 8 clearly shows that the reputation values of honest node gradually increase with the number of training rounds. In contrast, the reputation values of malicious node decrease rapidly with the number of malicious actions until reaching zero. These results demonstrate that our scheme's reputation mechanism can effectively distinguish between honest and malicious nodes, further enhancing system security and stability through dynamic adjustments of reputation values.

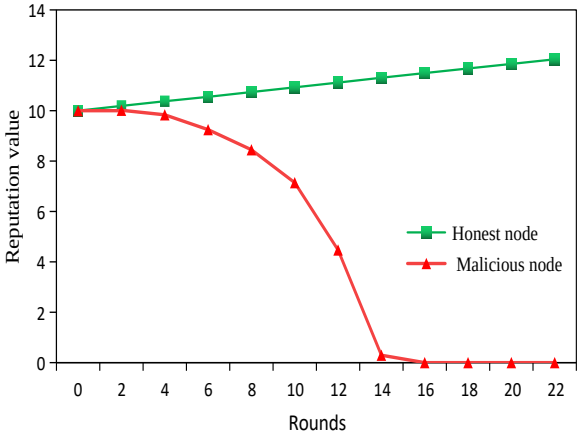

Figure 8: Reputation value change curves of typical honest nodes and malicious nodes

5.4. Scalability and Overhead Analysis

To address the practical feasibility of our framework in large-scale IoT environments, we provide a theoretical analysis of its scalability and operational overheads.

Scalability: The proposed framework supports horizontal scalability. As the number of IoT devices increases, the role separation mechanism ensures that the burden of model validation is distributed among validation nodes. The complexity of the reputation management algorithm is $O(N)$, where N is the number of participants. Since reputation updates are handled by smart contracts, the computational load is offloaded to the blockchain network. For a network scale of 100 to 1000 nodes, the communication overhead for model aggregation grows linearly. However, by utilizing a consortium blockchain with RBFT consensus, the system can handle thousands of transactions per second (TPS), which is sufficient for periodic FL model updates.

Computational Overhead: The primary computational costs arise from local model training and isolation forest-based anomaly detection. Local training is performed off-chain and depends on the device's capability. The anomaly detection, executed by validation nodes, has a time complexity of $O(t \cdot \psi)$, where t is the number of trees and ψ is the subsampling size. This is computationally efficient compared to distance-based methods like K-means ($O(N^2)$) or cryptographic methods used in homomorphic encryption schemes.

Communication Overhead: The communication cost consists of uploading model updates and blockchain transactions. For a standard CNN model (approx. 1.2MB parameters), the transmission delay is negligible on modern 4G/5G/WiFi networks. Blockchain transaction delays are primarily determined by the block generation time (typically 1-2 seconds in consortium chains) and consensus latency. Our choice of RBFT ensures that consensus latency remains low even as the number of validation nodes increases, making the framework feasible for time-sensitive IoT applications.

6. Conclusion

We presented BRITE-FL, a blockchain-based secure federated learning framework that addresses IoT device heterogeneity through role-based participation and mitigates malicious attacks via reputation token incentives. Simultaneously, the framework utilizes blockchain's decentralized characteristics and smart contract technology to achieve dynamic management and reputation assessment of node behavior, identifying and suppressing malicious nodes while incentivizing honest nodes to contribute continuously. Experimental results demonstrate that when facing data pollution and label flipping attacks, even with malicious node proportions as high as 30%, the framework can limit model performance degradation to within 5% and achieve over 90% malicious node identification rate, exhibiting significant attack resistance. This research provides a new technical approach to enhance the security and reliability of federated learning systems and lays a foundation for constructing robust distributed machine learning systems in complex IoT environments. Future work will focus on optimizing the framework's computational efficiency and scalability, exploring the introduction of privacy protection mechanisms, and extending the framework to other distributed learning scenarios.

References

[1] Juncen Zhu, Jiannong Cao, Divya Saxena, Shan Jiang, and Houda Ferradi. Blockchain-empowered federated learning: Challenges, solutions, and future directions. *ACM Computing Surveys*, 55(11):1–31, 2023.

[2] Yang Zhao, Jun Zhao, Mengmeng Yang, Teng Wang, Ning Wang, Lingjuan Lyu, Dusit Niyato, and Kwok-Yan Lam. Local differential privacy-based federated learning for internet of things. *IEEE Internet of Things Journal*, 8(11):8836–8853, 2020.

[3] Mansoor Ali, Hadis Karimipour, and Muhammad Tariq. Integration of blockchain and federated learning for internet of things: Recent advances and future challenges. *Computers & Security*, 108:102355, 2021.

[4] Yuzheng Li, Chuan Chen, Nan Liu, Huawei Huang, Zibin Zheng, and Qiang Yan. A blockchain-based decentralized federated learning framework with committee consensus. *IEEE Network*, 35(1):234–241, 2020.

[5] Dapeng Man, Fanyi Zeng, Wu Yang, Miao Yu, Jiguang Lv, and Yijing Wang. Intelligent intrusion detection based on federated learning for edge-assisted internet of things. *Security and Communication Networks*, 2021(1):9361348, 2021.

[6] Weishan Zhang, Qinghua Lu, Qiuyu Yu, Zhaotong Li, Yue Liu, Sin Kit Lo, Shiping Chen, Xiwei Xu, and Liming Zhu. Blockchain-based federated learning for device failure detection in industrial iot. *IEEE Internet of Things Journal*, 8(7):5926–5937, 2020.

[7] Jiaxiang Zhang, Yiming Liu, Xiaoqi Qin, Xiaodong Xu, and Ping Zhang. Adaptive resource allocation for blockchain-based federated learning in internet of things. *IEEE Internet of Things Journal*, 10 (12):10621–10635, 2023.

[8] Liang Gao, Li Li, Yingwen Chen, ChengZhong Xu, and Ming Xu. Fgfl: A blockchain-based fair incentive governor for federated learning. *Journal of Parallel and Distributed Computing*, 163:283–299, 2022.

[9] Unal Tatar, Yasir Gokce, and Brian Nussbaum. Law versus technology: Blockchain, gdpr, and tough tradeoffs. *Computer Law & Security Review*, 38:105454, 2020.

[10] H. Brendan Mcmahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. 2016.

[11] Stacey Truex, Ling Liu, Ka-Ho Chow, Mehmet Emre Gursoy, and Wenqi Wei. Ldp-fed: Federated learning with local differential privacy. In *Proceedings of the third ACM international workshop on edge systems, analytics and networking*, pages 61–66, 2020.

[12] Gaoyang Liu, Chen Wang, Xiaoqiang Ma, and Yang Yang. Keep your data locally: Federated-learning-based data privacy preservation in edge computing. *IEEE Network*, 35(2):60–66, 2021.

[13] Jingcheng Song, Weizheng Wang, Thippa Reddy Gadekallu, Jianyu Cao, and Yining Liu. Eppda: An efficient privacy-preserving data aggregation federated learning scheme. *IEEE Transactions on Network Science and Engineering*, 10(5):3047–3057, 2022.

[14] Zhe Yang, Kan Zheng, Kan Yang, and Victor C. M. Leung. A blockchain-based reputation system for data credibility assessment in vehicular networks. *IEEE*, 2017.

[15] Qipeng Xie, Siyang Jiang, Linshan Jiang, Yongzhi Huang, Zhihe Zhao, Salabat Khan, Wangchen Dai, Zhe Liu, and Kaishun Wu. Efficiency optimization techniques in privacy-preserving federated learning with homomorphic encryption: A brief survey. *IEEE Internet of Things Journal*, 11(14):24569–24580, 2024.

[16] Xin Wang, Haoji Zhang, Haoyu Wu, and Hongnian Yu. Dual-blockchain based multi-layer grouping federated learning scheme for heterogeneous data in industrial iot. *Blockchain: Research and Applications*, page 100195, 2024.

[17] Chunghun Baek, Sungwook Kim, Dongkyun Nam, and Jihoon Park. Enhancing differential privacy for federated learning at scale. *IEEE Access*, 9:148090–148103, 2021.

[18] Md Ashraf Uddin, Andrew Stranieri, Iqbal Gondal, and Venki Balasubramanian. A survey on the adoption of blockchain in iot: Challenges and solutions. *Blockchain: Research and Applications*, 2(2):100006, 2021.

[19] Noora Mohammed Al-Maslamani, Mohamed Abdallah, and Bekir Sait Ciftler. Reputation-aware multi-agent drl for secure hierarchical federated learning in iot. *IEEE Open Journal of the Communications Society*, 4:1274–1284, 2023.

[20] Umer Majeed, Latif U Khan, Sheikh Salman Hassan, Zhu Han, and Choong Seon Hong. Fl-incentivizer: Fl-nft and fl-tokens for federated learning model trading and training. *IEEE Access*, 11:4381–4399, 2023.

[21] Mohd Javaid, Abid Haleem, Ravi Pratap Singh, Shahbaz Khan, and Rajiv Suman. Blockchain technology applications for industry 4.0: A literature-based review. *Blockchain: Research and Applications*, 2 (4):100027, 2021.

[22] Yunhua He, Mingshun Luo, Bin Wu, Limin Sun, Yongdong Wu, Zhiquan Liu, and Ke Xiao. A game theory-based incentive mechanism for collaborative security of federated learning in energy blockchain environment. *IEEE Internet of Things Journal*, 10(24):21294–21308, 2023.

[23] Huaqun Guo and Xingjie Yu. A survey on blockchain technology and its security. *Blockchain: research and applications*, 3(2):100067, 2022.

[24] Zhilin Wang, Qin Hu, and Xu Zehui Xiong. Incentive mechanism design for joint resource allocation in blockchain-based federated learning. *IEEE Transactions on Parallel and Distributed Systems: A Publication of the IEEE Computer Society*, 34(5):1536–1547, 2023.

[25] Umer Majeed, Latif U. Khan, Sheikh Salman Hassan, Zhu Han, and Choong Seon Hong. Fl-incentivizer: Fl-nft and fl-tokens for federated learning model trading and training. *IEEE Access*, 11:4381–4399, 2023. doi: 10.1109/ACCESS.2023.3235484.

[26] Yunlong Lu, Xiaohong Huang, Yueyue Dai, Sabita Maharjan, and Yan Zhang. Blockchain and federated learning for privacy-preserved data sharing in industrial iot. *IEEE Transactions on Industrial Informatics*, 16(6):4177–4186, 2019.

[27] Muhammad Shayan, Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. Biscotti: A blockchain system for private and secure federated learning. *IEEE Transactions on Parallel and Distributed*

Systems, 32(7):1513–1525, 2021.

[28] Jun Li, Weiwei Zhang, Kang Wei, Guangji Chen, Feng Shu, Wen Chen, and Shi Jin. Blockchain-aided wireless federated learning: Resource allocation and client scheduling. *arXiv preprint arXiv:2406.00752*, 2024. URL https://arxiv.org/abs/2406.00752.

[29] Youyang Qu, Longxiang Gao, Yong Xiang, Shigen Shen, and Shui Yu. Fedtwin: Blockchain-enabled adaptive asynchronous federated learning for digital twin networks. *IEEE Network*, 36(6):183–190, 2022.

[30] Xuhan Zuo, Minghao Wang, Tianqing Zhu, Lefeng Zhang, Dayong Ye, Shui Yu, and Wanlei Zhou. Federated trustchain: Blockchain-enhanced llm training and unlearning. *arXiv preprint arXiv:2406.04076*, 2024.

[31] Zirun Zhao, Yi Sun, Ali Kashif Bashir, and Zhaowen Lin. Adadpfed: A differentially private federated learning algorithm with adaptive noise on non-iid data. *IEEE Transactions on Consumer Electronics*, 2023. doi: 10.1109/TCE.2023.3320122.

[32] Aditya Pribadi Kalapaaking, Ibrahim Khalil, Mohammad Saidur Rahman, Mohammed Atiquzzaman, Xun Yi, and Mahathir Almashor. Blockchain-based federated learning with secure aggregation in trusted execution environment for internet-of-things. *IEEE Transactions on Industrial Informatics*, 19(2):1703–1714, 2023. doi: 10.1109/TII.2022.3170348.

[33] Yang Zhao, Jun Zhao, Linshan Jiang, Rui Tan, Dusit Niyato, Zengxiang Li, Lingjuan Lyu, and Yingbo Liu. Privacy-preserving blockchain-based federated learning for iot devices. *IEEE Internet of Things Journal*, 8(3):1817–1829, 2021.

[34] Tao Wang, Bo Zhao, and Liming Fang. Flforest: Byzantine-robust federated learning through isolated forest. *IEEE Access*, 8:175215–175223, 2020.

[35] Guowen Dai, Jinjun Tang, Jie Zeng, Chen Hu, and Chuyun Zhao. Road network traffic flow prediction: A personalized federated learning method based on client reputation. *Computers and Electrical Engineering*, 120:109678, 2024.

[36] Jiawen Kang, Zehui Xiong, Dusit Niyato, Shengli Xie, and Junshan Zhang. Incentive mechanism for reliable federated learning: A joint optimization approach to combining reputation and contract theory. *IEEE Internet of Things Journal*, 6(6):10700–10714, 2019. doi: 10.1109/JIOT.2019.2940820.

[37] Wei Yang Bryan Lim, Zehui Xiong, Chunyan Miao, Dusit Niyato, Qiang Yang, Cyril Leung, and H. Vincent Poor. Hierarchical incentive mechanism design for federated machine learning in mobile networks. *IEEE Internet of Things Journal*, 7(10):9575–9588, 2020. doi: 10.1109/JIOT.2020.2985694.

[38] Bijun Wu and Oshani Seneviratne. Blockchain-based framework for scalable and incentivized federated learning. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 1761–1767, 2025.

[39] Ming Tang, Fu Peng, and Vincent WS Wong. A blockchain-empowered incentive mechanism for cross-silo federated learning. *IEEE Transactions on Mobile Computing*, 2024.

[40] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 eighth ieee international conference on data mining*, pages 413–422. IEEE, 2008.

[41] Sahand Hariri, Matias Carrasco Kind, and Robert J Brunner. Extended isolation forest. *IEEE transactions on knowledge and data engineering*, 33(4):1479–1489, 2019.

[42] Liang Cai, Qilei Li, and Xiubo Liang. Hyperchain application development fundamentals. In *Advanced Blockchain Technology: Frameworks and Enterprise-Level Practices*, pages 273–292. Springer, 2022.